

A Zero-Knowledge Argument of Knowledge of a Lean 4 Proof

RequestProject

Abstract

This document *is* the argument. It publishes a Pedersen commitment C to a digest of a Lean 4 source corpus, together with one Fiat–Shamir-compiled Okamoto transcript (a, z_1, z_2) proving knowledge of an opening of C . Anybody can check the five numbers below with a bignum library and any SHA-256 implementation; nothing about the committed corpus is disclosed by them. Every property of the protocol used here — completeness, special soundness, perfect hiding, binding, and honest-verifier zero knowledge — is a machine-checked theorem inside the very corpus that is committed to, and so is the assertion that the arithmetic printed in §6 is the protocol’s verification equation. The cryptography is entirely standard: Pedersen commitments, the Okamoto Σ -protocol, the Fiat–Shamir transform, a Merkle tree, and SHA-256.

1 The claim

I know the Lean 4 source corpus whose SHA-256 Merkle root, reduced mod q , is the value committed to in C : the corpus of this project, including the machine-checked proofs of completeness, special soundness, perfect hiding, binding and honest-verifier zero knowledge for the very protocol used to make this statement.

That sentence is not decoration: it is hashed, byte for byte, into the Fiat–Shamir challenge (§4), so the transcript below cannot be lifted and reused under a different claim.

What the corpus *does*, stated at the level of detail this document is willing to disclose: it is a Lean 4 development, built against Mathlib, about finite automata whose transition monoids are groups and about the arithmetic of finite simple groups; the part of it that is disclosed here is its cryptographic layer, the one described in §§2–7. Everything else — file names, file sizes, the number of files, statements, proofs — stays inside the commitment. By the hiding theorem of §7 the published numbers reveal none of it, not even to an adversary with unbounded computing power.

2 Public parameters

The protocol runs in the prime-order subgroup $G = \langle g \rangle \leq \mathbb{F}_p^\times$, where

$$q = 2^{1130} \cdot 3^{457} \cdot 5^{83} + 1, \quad p = 16q + 1, \quad g = 2^{16},$$

q is a 2048-bit prime, p is a 2052-bit prime, and g has order exactly q . A second generator $y \in G$ is derived by hashing a fixed public seed to the group, so that nobody — the prover included — knows $\log_g y$; that unknown logarithm is what makes the commitment binding. The script `docs/zkpop-params.py` regenerates all four numbers deterministically.

[illegible][illegible]

65536

8156381879568962919956241753249163371036478191969374003959354567
0046727877170159569342499521164755538261381005228347564438490884
1923870626480272525220219331809046160814011725401822741455934369
4854028198467912620480735190418554177448224327181116304961426255
4383809323291033950816525371916837914089892432161872324059211971
2618620959055306658448387475301706576796369451597814782018043299
8003005133622530955738584540402741670350244196347219972748572361
8508682963170294211657715362080797722771132740642478481782788978
8785204912608914022212595923604055866705052881463091898578154529
69617011354535603754722245068957456432433

The shape of $q - 1$ is not an accident. Both primalities are *proved inside Lean*, by Lucas certificates (`lucas_primality` in Mathlib, i.e. the classical Lucas–Pratt test): a witness w with $w^{n-1} = 1$ and $w^{(n-1)/r} \neq 1$ for every prime $r \mid n - 1$. Such a certificate needs the complete factorisation of $n - 1$, so q was searched for among the $\{2, 3, 5\}$ -smooth shifts, giving $q - 1 = 2^{1130} 3^{4575} 83$ and $p - 1 = 2^4 q$. The witnesses are 17 for q and 3 for p . The modular exponentiations these certificates need are performed by a square-and-multiply routine that is itself verified (`powMod`, with `powMod eq : powMod m a k = a^k % m`).

3 What is committed to

The corpus is condensed as follows.

1. Every *.lean file of the project, in sorted path order, contributes a leaf

$$\ell_i = \text{SHA256}(0\text{x}00 \parallel \text{path} \parallel 0\text{x}00 \parallel \text{SHA256}(\text{contents})).$$

2. Leaves are folded pairwise into a Merkle root with $\text{node}(l, r) = \text{SHA256}(0\text{x}01 \parallel l \parallel r)$, a lone last node being promoted unchanged. The two domain-separating tag bytes prevent a leaf from being reinterpreted as an internal node.
3. The root, read as a big-endian integer and reduced modulo q , is the committed scalar $w \in \mathbb{Z}/q\mathbb{Z}$.

The prover then draws a uniform blinding factor $s \in \mathbb{Z}/q\mathbb{Z}$ from the operating system’s entropy source and publishes the Pedersen commitment

$$C = g^w y^s \bmod p.$$

Because s is uniform and y generates G , the distribution of C is uniform on G *whatever w is*: the commitment is perfectly, i.e. information-theoretically, hiding. Because $\log_g y$ is unknown, opening C in two different ways would exhibit that logarithm: the commitment is computationally binding.

4 The protocol

Knowledge of the pair (w, s) is proved with the Okamoto Σ -protocol, the two-generator version of Schnorr’s identification protocol:

prover	verifier
$r_1, r_2 \leftarrow \mathbb{Z}/q\mathbb{Z}, \quad a = g^{r_1} y^{r_2}$	$\xrightarrow{a}$
	$\xleftarrow{c} \quad c \leftarrow \mathbb{Z}/q\mathbb{Z}$
$z_1 = r_1 + c w, \quad z_2 = r_2 + c s$	$\xrightarrow{z_1, z_2} \quad \text{check } g^{z_1} y^{z_2} = a C^c$

The Fiat–Shamir transform replaces the verifier’s coin flips by a hash, turning the interaction into the single message published here. The challenge is

$$c = \text{SHA256}(\text{RequestProject/ZKPoP/LeanProof/FiatShamir/v1} \parallel 0\text{x}00 \parallel \text{claim} \parallel 0\text{x}00 \parallel \langle p \rangle \parallel \langle q \rangle \parallel \langle g \rangle \parallel \langle y \rangle \parallel \langle C \rangle \parallel \langle a \rangle) \bmod q,$$

where $\langle \cdot \rangle$ is the 264-byte big-endian encoding and *claim* is the sentence of §1 in UTF-8.

5 The published argument

C (Pedersen commitment to the corpus digest)

```
1130632911894761205307900145161269691276090693301253326877958516
7183219469720309804540366196793354530243284223950915260214639438
9382711611858157834107026839307015436177991206216413057789802651
7950277582789406405733877099565769669670629139643097784538942916
6899955829061223544310875080069505987098970287748042364682131975
8668629448827087683172727101874366310516394734997965705802029354
1904546291772837331642905586073910525467144822071855931706779104
```

6391575832397465881584548993353573511401286087484912599719904413
5761023438921319094553855140448596604089434999843593954940596046
104493664542268373241630067241164642771983

a (first message)

4492585956774784808731373127699371081076290992446799961182163292
8327269048165533426978066572777726805291303375336454066781415611
9396674791568456558949660288897784625644309860030336390907624609
3462262826158562715212025676280002332130447995476954411009509048
9118686927168364523462726830626300163865534006346221587146714250
7627048385902672688548406163254042799776620938053895418333473254
4194460531233263441398235142993375619466767538577488100585364757
0879951751293597746609958377755508902251856953421843750721995782
7673124911093532102832655462705919743522816202524778844906524961
73243004138708320004626165672115451236025

c (Fiat–Shamir challenge, recomputable from the above)

90181885314238192395813514762326354068362262484443203979398489563132486113002

z_1 (response for the committed value)

1210762346797476369734961301849631780467385733747487667652045011
5823418235190101692523188879310823900674019470100364265271618009
539636321747667577009701300

z_2 (response for the blinding factor)

2528150926632021805898988220199307104382763750407680243145047098
0373133710514713297492624746622192243617451718118244129106855337
9717261997107098285519182065518147859867077358886791090223798316
430825968266071956294589664715961015861

These five numbers, together with the four parameters and the claim string, are the entire public output; they are also stored in machine-readable form in `docs/zkpop-artifact.json`. The secrets w , s , r_1 , r_2 were never written to disk.

6 How to check it

A verifier performs exactly five checks:

1. $0 < C < p$ and $C^q \equiv 1 \pmod{p}$ (so C lies in the prime-order subgroup);
2. $0 < a < p$ and $a^q \equiv 1 \pmod{p}$;
3. $z_1 < q$ and $z_2 < q$;
4. c is the SHA-256 value displayed in §4;
5. $g^{z_1}y^{z_2} \equiv aC^c \pmod{p}$.

Two independent implementations of these checks accompany the document: `docs/zkpop-verify.py` (30 lines of standard-library Python, using the operating system’s SHA-256) and `RequestProject.ZKPoP.verifyNI` (inside Lean, using the SHA-256 implementation of `RequestProject.Sha256`, which is itself checked against the FIPS 180-4 test vectors). Both accept the transcript above; the Lean-side check is recorded as the theorem `RequestProject.ZKPoP.Published.published_verifies`.

7 What is proved in Lean

Every statement in the table below is a theorem of the committed corpus, machine-checked by Lean 4 against Mathlib, with no `sorry` and no added axioms.

property	theorem	file
q, p prime (Lucas certificates)	<code>qParam_prime, pParam_prime</code>	<code>ZKPoPGroup</code>
g has order q	<code>gElt_orderOf</code>	<code>ZKPoPGroup</code>
$y = g^t$ for a unit t nobody knows	<code>exists_dlog_y</code>	<code>ZKPoPGroup</code>
verified square-and-multiply	<code>powMod_eq</code>	<code>ZKPoPModArith</code>
SHA-256 matches FIPS 180-4 vectors	<code>sha256_abc</code> etc.	<code>Sha256</code>
completeness	<code>ped_completeness</code>	<code>ZKPoPPedersen</code>
special soundness (extracts an opening)	<code>ped_specialSoundness_prime</code>	<code>ZKPoPPedersen</code>
perfect hiding	<code>ped_hiding, ped_hiding_range_eq</code>	<code>ZKPoPPedersen</code>
binding computes $\log_g y$	<code>ped_binding_extracts_dlog</code>	<code>ZKPoPPedersen</code>
honest-verifier zero knowledge	<code>ped_shvzk, ped_shvzk_range_eq</code>	<code>ZKPoPPedersen</code>
published verifier = abstract verifier	<code>verifyNI_sound</code>	<code>ZKPoPLeanProof</code>
knowledge extraction	<code>knowledge_extraction</code>	<code>ZKPoPLeanProof</code>
this transcript verifies	<code>published_verifies</code>	<code>ZKPoPPublished</code>
this transcript is an accepting one	<code>published_is_accepting_transcript</code>	<code>ZKPoPPublished</code>

Three of these deserve a word.

Theorem 1 (Special soundness, `ped_specialSoundness_prime`). *If (a, c_1, z_1, z_2) and (a, c_2, z'_1, z'_2) are two accepting transcripts for the same C and the same first message a , with $c_1 \neq c_2$, then*

$$\hat{w} = \frac{z_1 - z'_1}{c_1 - c_2}, \quad \hat{s} = \frac{z_2 - z'_2}{c_1 - c_2}$$

satisfy $g^{\hat{w}}y^{\hat{s}} = C$. The extractor therefore turns any prover that answers two challenges into an algorithm that outputs an opening; that is what “argument of knowledge” means.

Theorem 2 (Perfect hiding, `ped_hiding`). *For any two values w, w' there is a bijection e of $\mathbb{Z}/q\mathbb{Z}$ with $g^{w'}y^{e(s)} = g^wy^s$ for all s . Commitments to different corpora are therefore identically distributed: C leaks nothing, unconditionally.*

Theorem 3 (The printed arithmetic is the protocol, `verifyNI_sound`). *If `verifyNI` accepts a numeric transcript, then the residues C and a are genuine elements of the prime-order group $\langle g \rangle \leq \mathbb{F}_p^\times$ and, as such, satisfy the abstract verification predicate `Base.PedAccepts` at the Fiat–Shamir challenge.*

The last theorem is what stitches the two halves of the document together: without it, the security theorems would be about an idealised protocol and the numbers in §5 about a computation that merely resembles it.

Remark 4 (Axioms). The Lean proofs use only `propext`, `Classical.choice` and `Quot.sound`, plus `Lean.ofReduceBool` and `Lean.trustCompiler` wherever a large computation is discharged by compiled evaluation (the primality certificates, the SHA-256 test vectors, and the verification of the published transcript).

8 What this does, and does not, establish

Established, unconditionally. C reveals nothing whatsoever about the corpus; the transcript (a, c, z_1, z_2) is simulatable from C alone, so it reveals nothing either. This is the content of the hiding and zero-knowledge theorems and it depends on no computational assumption.

Established, under standard assumptions. That the party who produced the transcript *knows* an opening (w, s) of C , and that it cannot later open C to a different value. The assumptions are the usual two: hardness of the discrete logarithm problem in the 2048-bit prime-order group $\langle g \rangle \leq \mathbb{F}_p^\times$, and the Fiat–Shamir heuristic — SHA-256 modelled as a random oracle, in which model a cheating prover must guess a 256-bit challenge, so that its success probability per hash query is about 2^{-256} . (In the interactive protocol, where the challenge is uniform in $\mathbb{Z}/q\mathbb{Z}$, special soundness gives error $1/q \approx 2^{-2048}$.)

Not established. That the committed corpus *compiles*, or that it proves any particular mathematical statement. Nothing in a commitment can certify that, because the verifier never sees the corpus. Two honest routes exist and are worth naming: opening the commitment later (the recipient recomputes the Merkle root and checks it against C and the revealed (w, s) , which turns this document into a timestamped, binding escrow of the development), or a general-purpose zero-knowledge proof system applied to the Lean type-checker, which is a far heavier construction and is not attempted here.

Parameter sizes. p has 2052 bits, comparable with standard finite-field Diffie–Hellman parameters; the subgroup order q has 2048 bits, far above the usual 256, which costs a little speed and buys nothing beyond the point where generic attacks are infeasible. The binding assumption is only as good as discrete logarithms in $\mathbb{F}_p^\times$; hiding, being information-theoretic, is unaffected by any advance in that problem.

9 Reproducing this document

- `python3 docs/zkpop-params.py` — regenerate p, q, g, y from their public description.
- `python3 docs/zkpop-verify.py` — check the transcript with the standard library only.
- `lake exe zkpopTranscript` — re-run the prover (a fresh blinding factor produces a different, equally valid transcript, and rewrites `docs/zkpop-artifact.{json, tex}` and `RequestProject/ZKPoPPublished.lean`).
- `lake build RequestProject.ZKPoPPublished` — have Lean re-verify the published transcript and everything it depends on.
- `tectonic docs/zkpop-lean-proof-argument.tex` — rebuild this PDF.

References

- [1] C.-P. Schnorr, *Efficient signature generation by smart cards*, Journal of Cryptology 4 (1991), 161–174.

- [2] T. P. Pedersen, *Non-interactive and information-theoretic secure verifiable secret sharing*, CRYPTO '91, LNCS 576, 129–140.
- [3] T. Okamoto, *Provably secure and practical identification schemes and corresponding signature schemes*, CRYPTO '92, LNCS 740, 31–53.
- [4] A. Fiat and A. Shamir, *How to prove yourself: practical solutions to identification and signature problems*, CRYPTO '86, LNCS 263, 186–194.
- [5] R. C. Merkle, *A digital signature based on a conventional encryption function*, CRYPTO '87, LNCS 293, 369–378.
- [6] V. R. Pratt, *Every prime has a succinct certificate*, SIAM Journal on Computing 4 (1975), 214–220.
- [7] National Institute of Standards and Technology, *Secure Hash Standard*, FIPS PUB 180-4, 2015.